

Project 4: Simple Router (layer 3)

This project is a localized version of Stanford's Simple Router project. You can refer to the original version if you want: <http://www.scs.stanford.edu/09au-cs144/lab/router.html>

In this project, you will implement a simple software/virtual router with a static forwarding table. All machines in the network have a fixed IP and a MAC address. You need to write the code for (1) packet forwarding based on the forwarding table and (2) layer-2/layer-3 protocols so that the packets are delivered to the correct interface.

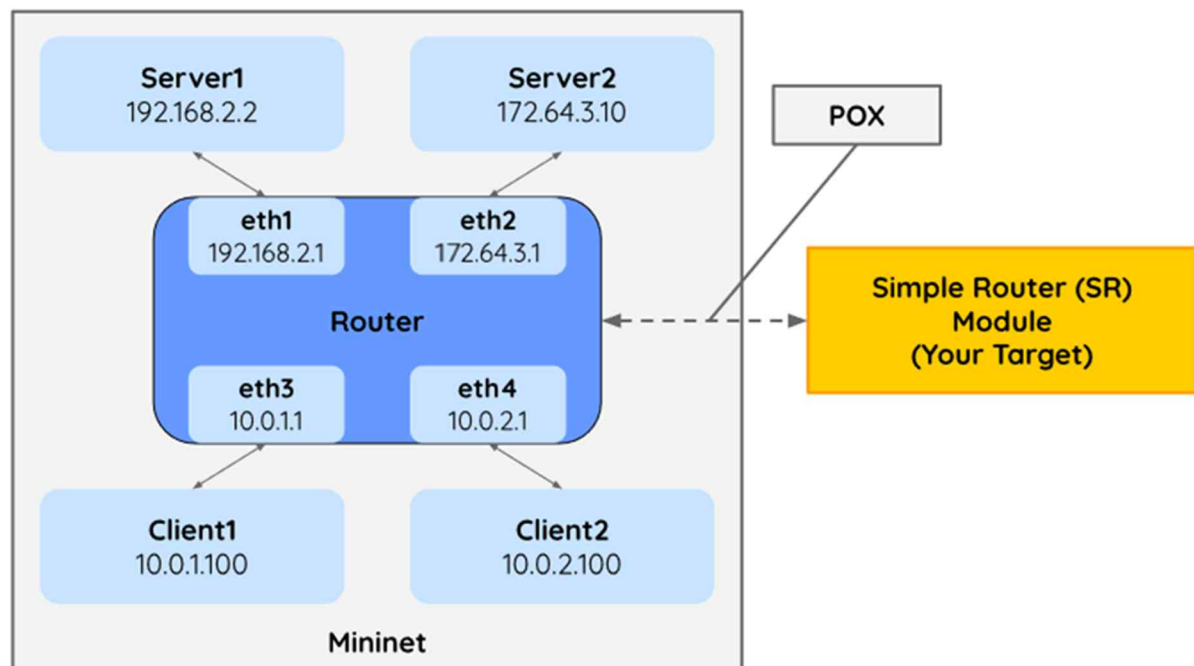


Figure 1: Virtual Network Environment for the project

Virtual Network Environment

We are going to use a virtual network emulated on **Mininet** built at Stanford University. The virtual network above, for example, hosts two servers and two clients that are connected to the virtual/software router directly. Servers, clients, and routers in the network can send and receive Ethernet frames just like in a real network.

- Simple Router (SR) module is where the code for router logic runs. It is attached to the Router via the **POX** internal communication interface (see Figure 1). The environment that we provide already sets up POX and Mininet properly, so you don't need to know anything about the setup. Please just focus on writing the core router functionalities.
- Note that a typical router has multiple interfaces (e.g., 4 interfaces in Figure 1 above) where each interface is assigned an IP address. Assume that servers and clients implement all protocols properly – you need to develop the code only for the router. Be careful about errors (e.g., wrong checksum), as packets with errors would be dropped at end hosts.

Pre-requisites:

(We borrow the text below from the corresponding EE323 assignment at KAIST)

To do the project, you should fully understand 1) how the router works, and 2) how each underlying protocol (Ethernet, IP, ICMP, and ARP) works. Here we provide a brief explanation, but you must check what fields you should fill in each protocol.

1. General Forwarding Logic

Given a raw Ethernet frame, if the frame contains an IP packet that is not destined for one of our interfaces:

1. Sanity-check the packet (meets minimum length and has correct checksum).
2. Check the IP. If an IP is on the blacklist, drop (ignore) the packet.
3. Decrement the TTL by 1, and recompute the packet checksum over the modified header.
4. Find out which entry in the routing table has the longest prefix match (LPM) with the destination IP address.
5. Check the ARP cache for the next-hop MAC address corresponding to the next-hop IP. If it's there, send it. Otherwise, send an ARP request for the next-hop IP, and add the packet to the queue of packets waiting on this ARP request.

Obviously, this is a very simplified version of the forwarding process, and the low-level details follow. For example, if an error occurs in any of the above steps, you will have to send an ICMP message back to the sender notifying them of an error. You may also get an ARP request or reply, which has to interact with the ARP cache correctly.

2. Protocols to Understand

A. Ethernet

You are given a raw Ethernet frame and have to send raw Ethernet frames. You should understand the source and destination MAC addresses and the idea that we forward a packet one hop by changing the destination MAC address of the forwarded packet to the MAC address of the next hop's incoming interface.

B. Internet Protocol version 4 (IPv4)

We will use the term IP as IPv4 for the entire document. We do not consider IPv6 here. Before operating on an IP packet, you should verify its checksum and make sure it meets the minimum length of an IP packet. You should understand how to find the longest prefix match of a destination IP address in the routing table, as described above. If you determine that a datagram should be forwarded, you should correctly decrement the TTL field of the header and recompute the checksum over the changed header before forwarding it to the next hop.

C. Address Resolution Protocol (ARP)

ARP is needed to determine the next-hop MAC address that corresponds to the destination IP address stored in the routing table. Without the ability to generate an ARP request and process ARP replies, your router would not be able to fill out the destination MAC address field of the raw Ethernet frame you are sending over the outgoing interface. Similarly, without the ability to process ARP requests and generate ARP replies, no other router could send your router Ethernet frames. Therefore, your router must generate and process ARP requests and replies.

To lessen the number of ARP requests sent out, you are required to cache ARP replies. Cache entries should time out after 15 seconds to minimize staleness. The provided ARP cache class already times the entries out for you.

When forwarding a packet to a next-hop IP address, the router should first check the ARP cache for the corresponding MAC address before sending an ARP request. In the case of a cache miss, an ARP request should be sent to a target IP address about once every second until a reply comes in. If the ARP request is sent five times with no reply, an ICMP destination host unreachable is sent back to the source IP as stated above. The provided ARP request queue will help you manage the request queue.

In the case of an ARP request, you should only send an ARP reply if the target IP address is one of your router's IP addresses. In the case of an ARP reply, you should only cache the entry if the target IP address is one of your router's IP addresses.

Note that ARP requests are sent to the broadcast MAC address (ff-ff-ff-ff-ff-ff). ARP replies are sent directly to the requester's MAC address.

D. Internet Control Message Protocol (ICMP)

An IP packet may contain an ICMP message. ICMP is used to send control information back to the sending host. You will need to properly generate the following ICMP messages (including the ICMP header checksum) in response to the sending host under the following conditions (you don't need to implement other types of messages in this project):

- **Echo reply (type 0, code 0)**

Send in response to an echo request (ping) to one of the router's interfaces. (This is only for echo requests to any of the router's IPs. An echo request sent elsewhere should be forwarded to the next-hop address as usual.)

- **Destination unreachable (type 3, code 0)**

Send if there is a non-existent route to the destination IP (no matching entry in the routing table when forwarding an IP packet).

- **Destination host unreachable (type 3, code 1)**
Send if five ARP requests were sent to the next-hop IP without a response.
- **Port unreachable (type 3, code 3)**
Send if an IP packet containing a UDP or TCP payload is sent to one of the router's interfaces. This is needed for traceroute to work.
- **Time exceeded (type 11, code 0)**
Sent if an IP packet is discarded during processing because the TTL field is 0.
This is also needed for traceroute to work.

The source address of an ICMP message can be the source address of any of the incoming interfaces, as specified in RFC 792. For type 3 and type 11 responses, don't forget to add the original IP header + 8bytes (28bytes) to the data field.

As mentioned above, the only incoming ICMP message destined towards the router's IPs that you have to explicitly process are ICMP echo requests.

Getting Started

Getting Started

1. Introduction

We provide a Virtual Machine (VM) image in which all required programs and environments are already set up for this project. This means that you will run an Ubuntu operating system on top of your existing operating system using VM software, VirtualBox.

You need to use your own machine to run VirtualBox. Before submission, please make sure that your project runs correctly on the provided VM. You may set up your own environment instead of using the provided VM image, but you do so at your own risk. Grading will be conducted in the same environment as the provided VM image, and we will not accept excuses caused by differences in running environments.

2. Install VirtualBox

Install the latest version of VirtualBox that matches your operating system and CPU architecture. <https://www.virtualbox.org/wiki/Downloads>

3. Download and Import the VM Image

Download the appropriate VirtualBox VM image from ETL for your CPU architecture. Use "SimpleRouter_public.ova" for Intel/AMD x86-64 machines, and "SimpleRouter_apple.ova" for Apple Silicon Macs, such as M1/M2/M3/M4 laptops.

Import the OVA file into VirtualBox. You may change the VM settings, such as memory, CPU, and network configuration, but please keep in mind that your work will be evaluated on the TA's machine.

The VM account information is as follows:

For Intel/AMD x86-64 CPUs:

- Ubuntu 22.04. (Do not upgrade the operating system.)
- ID: student, PW: 1234

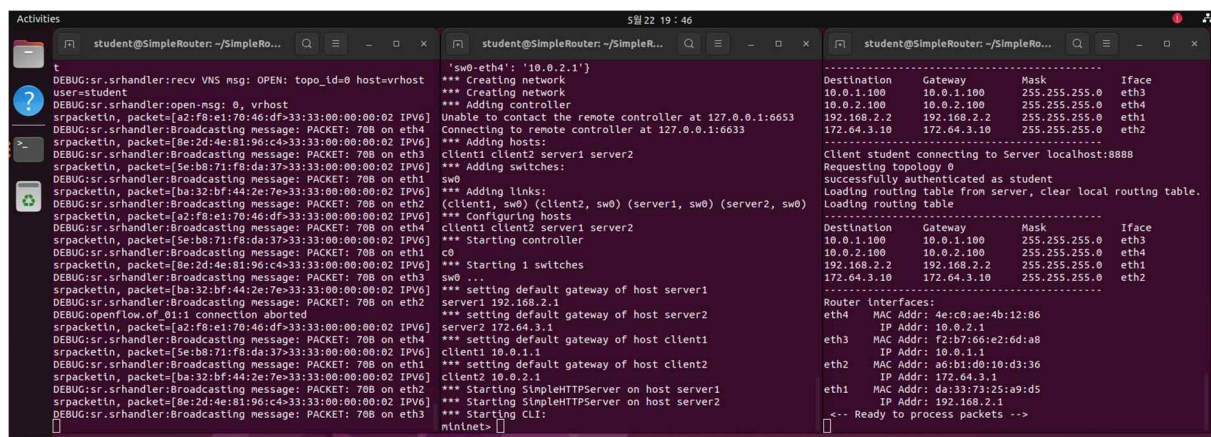
For ARM64 CPUs / Apple Silicon Macs:

- Ubuntu 24.04. (Do not upgrade the operating system.)
- ID: student, PW: 1234

The TA code and test cases run identically on both OVA files. However, to avoid unidentified exception cases, please specify in your README file which OVA file you used.

4. Check out the starter code

You will be able to check the **SimpleRouter** directory at the home directory (~/.SimpleRouter). You can check out the reference binary (program) **sr_solution**. In order to execute it, you should first set POX and Mininet in each terminal. **Be careful that each program should be executed in the exact order – wait sufficient time for each program to launch.**



The image shows three terminal windows side-by-side, all with the prompt 'student@SimpleRouter: ~/SimpleRouter...'. The leftmost terminal shows a series of DEBUG messages from 'sr.handler' and 'sr.packetin', indicating network packet processing. The middle terminal shows the execution of 'run_pox.sh', which includes commands to create a network, add controllers, add hosts, add switches, and start the controller. The rightmost terminal shows the execution of 'run_mininet.sh', which includes commands to load the routing table, request topology, and start the mininet environment. The output of 'run_mininet.sh' includes a table of destination, gateway, mask, and interface, and a list of router interfaces with their MAC and IP addresses.

```
student@SimpleRouter: ~/SimpleRouter...
DEBUG:sr.handler:recv VNS msg: OPEN: topo_id=0 host=vrhost
user=student
DEBUG:sr.handler:open-msg: 0, vrhost
sr.packetin, packets=[a2:f8:e1:7b:46:df>33:33:00:00:00:02 IPV6]
DEBUG:sr.handler:Broadcasting message: PACKET: 708 on eth4
sr.packetin, packets=[8e:2d:4e:81:96:c4>33:33:00:00:00:02 IPV6]
DEBUG:sr.handler:Broadcasting message: PACKET: 708 on eth3
sr.packetin, packets=[5e:b8:71:f8:da:37>33:33:00:00:00:02 IPV6]
DEBUG:sr.handler:Broadcasting message: PACKET: 708 on eth4
sr.packetin, packets=[ba:32:bf:44:2e:7e>33:33:00:00:00:02 IPV6]
DEBUG:sr.handler:Broadcasting message: PACKET: 708 on eth1
sr.packetin, packets=[8e:2d:4e:81:96:c4>33:33:00:00:00:02 IPV6]
DEBUG:sr.handler:Broadcasting message: PACKET: 708 on eth2
sr.packetin, packets=[a2:f8:e1:7b:46:df>33:33:00:00:00:02 IPV6]
DEBUG:sr.handler:Broadcasting message: PACKET: 708 on eth3
sr.packetin, packets=[5e:b8:71:f8:da:37>33:33:00:00:00:02 IPV6]
DEBUG:sr.handler:Broadcasting message: PACKET: 708 on eth4
sr.packetin, packets=[ba:32:bf:44:2e:7e>33:33:00:00:00:02 IPV6]
DEBUG:sr.handler:Broadcasting message: PACKET: 708 on eth1
sr.packetin, packets=[8e:2d:4e:81:96:c4>33:33:00:00:00:02 IPV6]
DEBUG:sr.handler:Broadcasting message: PACKET: 708 on eth2
DEBUG:openflow.of_01:1 connection aborted
sr.packetin, packets=[a2:f8:e1:7b:46:df>33:33:00:00:00:02 IPV6]
DEBUG:sr.handler:Broadcasting message: PACKET: 708 on eth4
sr.packetin, packets=[5e:b8:71:f8:da:37>33:33:00:00:00:02 IPV6]
DEBUG:sr.handler:Broadcasting message: PACKET: 708 on eth1
sr.packetin, packets=[ba:32:bf:44:2e:7e>33:33:00:00:00:02 IPV6]
DEBUG:sr.handler:Broadcasting message: PACKET: 708 on eth2
sr.packetin, packets=[8e:2d:4e:81:96:c4>33:33:00:00:00:02 IPV6]
DEBUG:sr.handler:Broadcasting message: PACKET: 708 on eth3
DEBUG:sr.handler:Broadcasting message: PACKET: 708 on eth3

*** Creating network
*** Creating network
*** Adding controller
Unable to contact the remote controller at 127.0.0.1:6653
Connecting to remote controller at 127.0.0.1:6633
*** Adding hosts:
client1 client2 server1 server2
*** Adding switches:
sw0
*** Adding links:
(client1, sw0) (client2, sw0) (server1, sw0) (server2, sw0)
*** Configuring hosts
client1 client2 server1 server2
*** Starting controller
c0
*** Starting 1 switches
sw0
*** setting default gateway of host server1
server1 192.168.2.1
*** setting default gateway of host server2
server2 172.64.3.1
*** setting default gateway of host client1
client1 10.0.1.1
*** setting default gateway of host client2
client2 10.0.2.1
*** Starting SimpleHTTPServer on host server1
*** Starting SimpleHTTPServer on host server2
*** Starting CLI:
mininet>

-----
Destination Gateway Mask Iface
10.0.1.100 10.0.1.100 255.255.255.0 eth3
10.0.2.100 10.0.2.100 255.255.255.0 eth4
192.168.2.2 192.168.2.2 255.255.255.0 eth1
172.64.3.10 172.64.3.10 255.255.255.0 eth2
-----
Client student connecting to Server localhost:8888
Requesting topology 0
successfully authenticated as student
Loading routing table from server, clear local routing table.
Loading routing table
-----
Destination Gateway Mask Iface
10.0.1.100 10.0.1.100 255.255.255.0 eth3
10.0.2.100 10.0.2.100 255.255.255.0 eth4
192.168.2.2 192.168.2.2 255.255.255.0 eth1
172.64.3.10 172.64.3.10 255.255.255.0 eth2
-----
Router interfaces:
eth4 MAC Addr: 4e:c0:ae:4b:12:86
IP Addr: 10.0.2.1
eth3 MAC Addr: f2:b7:66:e2:6d:a8
IP Addr: 10.0.1.1
eth2 MAC Addr: a6:b1:d0:10:d3:36
IP Addr: 172.64.3.1
eth1 MAC Addr: da:33:73:12:5a:9d:5
IP Addr: 192.168.2.1
<- Ready to process packets ->
```

Figure 2: Executing starter code in three terminals

<At terminal 1>

```
~/SimpleRouter$ ./run_pox.sh
```

<At terminal 2>

```
~/SimpleRouter$ sudo ./run_mininet.sh
```

<At terminal 3>

```
~/SimpleRouter$ ./sr_solution
```

```
mininet> client1 ping -c 3 192.168.2.2
PING 192.168.2.2 (192.168.2.2) 56(84) bytes of data:
64 bytes from 192.168.2.2: icmp_seq=1 ttl=63 time=240 ms
64 bytes from 192.168.2.2: icmp_seq=2 ttl=63 time=115 ms
64 bytes from 192.168.2.2: icmp_seq=3 ttl=63 time=74.3 ms

--- 192.168.2.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 74.330/143.063/240.220/70.643 ms
mininet>
```

Figure 3: Executing ping request from client1 to server1

Also, you can test your Simple Router on the Mininet (terminal 2). For example,

```
mininet> client1 ping -c 3 192.168.2.2
```

Will generate a ping (ICMP type0) message from client1 to the destination 192.168.2.2 (server1) three times (-c option). As an example, you can generate packets in order of

```
mininet> [source] [command] [options] [destination]
```

Where commands can be one of (ping, traceroute, wget). You can check what options are available for each command by typing a command without options and destination.

You can close each program with CTRL+C (for POX and Simple Router) or CTRL+D (for Mininet). If you fail the connectivity (due to a server down, Internet connection problems, etc.), close all existing terminals and kill processes using the command:

```
~/SimpleRouter$ ./killall.sh
```


You can build and execute your Simple Router in the **router** directory with:

```
~/SimpleRouter/router$ make
```

```
~/SimpleRouter/router$ ./sr
```

Also, you can generate a PCAP file by setting command-line arguments to log packets (usually for debugging purposes).

```
~/SimpleRouter/router$ ./sr -l filename.pcap
```

To analyze PCAP files, use Wireshark. Using the PCAP files, you can test and debug your code. Do not submit PCAP files. Instead, we will generate PCAP files and use them to grade your implementation integrity.

Data Structures

1. The Router (sr_router.h)

The full context of the router is housed in the struct `sr_instance` (sr_router.h).

`sr_instance` contains the information about the topology the router is operating for and the routing table and the list of interfaces.

2. Interfaces (sr_if.c/h)

After connecting, the server will send the client the hardware information for that host.

The stub code uses this to create a linked list of interfaces in the router instance at member `if_list`. Utility methods for handling the interface list can be found at `sr_if.c/h`.

3. The Routing Table (sr_rt.c/h)

The routing table in the stub code is read on from a file (default filename "rtable", can be set with command-line option `-r`) and stored in a linked list of routing entries in the current routing instance (member `routing_table`).

4. The ARP Cache and ARP Request Queue (sr_arpcache.c/h)

You will need to add ARP requests and packets waiting on responses to those ARP requests to the ARP request queue. When an ARP response arrives, you will have to remove the ARP request from the queue and place it onto the ARP cache, forwarding any packets waiting on that ARP request. Pseudocode for these operations is provided in `sr_arpcache.h`. The base code already creates a thread that times out ARP cache entries 15 seconds after they are added for you. You must fill out the `sr_arpcache_sweepreqs` function in `sr_arpcache.c` that gets called every second to iterate through the ARP request queue and re-send ARP requests if necessary. Pseudocode for this is provided in `sr_arpcache.h`.

5. ICMP Data Structure (sr_protocol.h)

You will find the ICMP header data structure in `sr_protocol.h`. Use appropriate headers (`struct sr_icmp_t0`, `sr_icmp_t3`, `sr_icmp_t11`) for type 0, 3, and 11 messages.

What Should I Do?

Your job is to implement three functions on two C files in the **router** directory.

- `sr_router.c`: `sr_handlepacket()`, `ip_black_list()`
- `sr_arpcache.c`: `sr_arpcache_handle_arpreq()`

Your router receives a raw Ethernet frame and sends raw Ethernet frames when sending a reply to the sending host or forwarding the frame to the next hop.

1. void sr_handlepacket(struct sr_instance *sr, uint8_t *packet, unsigned int len, char *interface)

This function is called by the router each time a packet is received. The "packet" argument points to the packet buffer which contains the full packet including the ethernet header. The name of the **receiving** interface is passed into the method as well.

An incoming IP packet may be destined for one of your router's IP addresses, or it may be destined elsewhere. If it is sent to one of your router's IP addresses, you should take the following actions, consistent with the section on protocols above:

- If the packet is an ICMP echo request and its checksum is valid, send an ICMP echo reply to the sending host.
- If the packet contains a TCP or UDP payload, send an ICMP port unreachable to the sending host.
- Otherwise, ignore the packet.

Packets destined elsewhere should be forwarded using your normal forwarding logic.

2. int ip_black_list(struct sr_ip_hdr *iph)

This function is called by sr_handlepacket() to filter out the packets that contain blacklisted IP, working as a firewall. You should block both inbound and outbound packets. Target IP and the corresponding mask are given in the code. If you successfully block, print the message: "[IP blocked] %s\n" (does not contain "") where %s contains the blocked target IP address in the Dotted-Decimal Notation (notation example: 172.0.0.1).

3. void sr_arpcache_sweepreqs(struct sr_instance *sr)

The project requires you to send an ARP request once a second until a reply comes back or we have sent five requests. This function is defined in sr_arpcache.c and called every second, and you should add code that iterates through the ARP request queue and re-sends any outstanding ARP requests that haven't been sent in the past second. If an ARP

request has been sent 5 times with no response, an ICMP packet with "destination host unreachable" should be sent to all the senders of held packets that were waiting on a reply to this ARP request.

4. Cautions & Tips (Not in particular order)

- In this assignment, when you create a packet, always set IP Time-To-Live to INIT_TTL (255). DON'T decrement the TTL value of the packet that you just created.
- You should not free the buffer given to you in `sr_handlepacket` (this is why the buffer is labeled as being "lent" to you in the comments). However, when you call `sr_send_packet()` for sending a packet, you are responsible for doing correct memory management on the buffers (that is, `sr_send_packet()` will not call `free` on the buffers that you pass it).
- Double-check if your program does not make meaningless outputs; print only blacklist messages (and check again if your blacklist message is correct!).
- ARP caches should be properly handled; their creation, recalls, and destruction.
- We suggest implementing in order of: `sr_handlepacket()` → `sr_arpcache_sweepreqs()` → `ip_black_list()`. Implement ARP-related stuff first to make other protocols work.
- Understand what is different between inbound and outbound packets.

How to debug?

1. Debugging functions

We provide you with some basic debugging functions in `sr_utils.c/h`. Feel free to use them to print out network header information from your packets. **Do not leave any functions that print messages out to STDOUT when you submit.** Below are some functions that you may find useful:

- `print_hdrs(uint8_t *buf, uint32_t length)`: Prints out all possible headers starting from the Ethernet header in the packet.
- `print_addr_ip_int(uint32_t ip)`: Prints out a formatted IP address from a `uint32_t` ip. Make sure you are passing the IP address in the correct byte ordering

2. GDB

You may have experienced a lot of `malloc()`, `calloc()`, and `free()` functions during previous projects and courses. However, dealing with them is always difficult - `SEGFAULT` will make you angry (if not, why are you reading this? :D). Use GDB to check where did the segmentation fault occur:

```
~/SimpleRouter/router$ gdb ./sr
```

```
(gdb) run
```

And make some inputs to the Mininet that invoke `SEGFAULT`.

3. Generating PCAP Logs

As mentioned above, PCAP files are useful for debugging. They parse packet logs and show details of each protocol. You can also examine the checksums (FYI, you should be aware that wrong checksums may lead to no response from clients/servers).

Useful Functions

1. Handling Endianness ([Reference](#))

You should be familiar with handling endianness: between network byte order and host byte order.

- `ntohl(uint32_t hostlong)`: converts the unsigned integer from host byte order to network byte order.
- `ntohl(uint32_t netlong)`: converts the unsigned integer from network byte order to host byte order.
- `htons(uint16_t hostlong)`: converts the unsigned short from network byte order to host byte order.
- `ntohs(uint16_t netlong)`: converts the unsigned short from network byte order to host byte order.

2. Handling IP Address [\(Reference\)](#)

You can manually convert IP addresses between network byte order (unsigned integer) and dotted-decimal formation (ex. "192.168.0.1"). You can utilize the following functions:

- `inet_ntoa(struct in_addr in)`: converts the IP address to dotted-decimal format.
- `inet_aton(const char *cp, struct in_addr *inp)`: converts the dotted-decimal format to IP address.

3. Calculating Checksum

You do not need to implement checksum calculation. The checksum can be easily calculated by `cksum()` function, but be careful to clear the checksum field before calculating it. You can find some examples utilizing checksum calculation with existing code at `sr_handlepacket()`.

Grading Criteria

- Integrity of the submission file - 5 pts
 - Submission file format, all required files, no modification of macro.h/Makefile, no additional files, etc)
- Content of readme.pdf - 10 pts
 - Submit the readme.pdf that contains the captured image of execution results for each grading criteria. The report will not be explicitly graded to the numerical value, but we will use it when there are any issues with grading.
- Coding style and compilation issues - 5 pts
 - Proper indentation and comments, proper variable naming, remove unnecessary global variables, no compilation warnings in, etc.
 - Please note that you will get zero credit if compilation fails. We will not allow you to fix any errors in your code once it's submitted.
- ARP – 16 pts
 - All subsequent test cases should work properly (i.e. if you fail ARP, then you may get a zero score for the entire project 4).
 - Broadcast – 4 pts
 - Reply unicast – 4 pts
 - Cache – 4 pts
 - Evict – 4 pts
- ICMP – 24 pts
 - Echo reply – 4 pts
 - Network unreachable – 4 pts
 - Host unreachable – 4 pts
 - Port unreachable – 4 pts
 - TTL exceeded – 4 pts
 - ICMP checksum correctness – 4 pts
- IP – 24 pts
 - TTL decrement – 4 pts
 - Initial maximum TTL – 4 pts
 - Forwarding – 16 pts
- Firewall – 16 pts

- Inbound – 4 pts
- Outbound – 4 pts
- Inbound log – 4 pts
- Outbound log – 4 pts
- **Important:** you will get a zero score if 'make' error(s) prevents us from building the binary. So, make sure to test with

`~/SimpleRouter/router$ make dist-clean; make`

before submission. Make sure that the clean build is working fine.

- **Important:** If your Simple Router binary has STDOUT output problems (such as adding any unnecessary messages), we will deduct **20%** of your score that you will get when the formatting problem is solved.
- We will run your submission 5 times and will give you the highest score.
- We will set a 30-second limit per test case first, and re-run those with a 3-minute limit that failed due to the timeout on the first trial. (There will be no penalty for failing a 30 seconds test but passing a 3-minute test.) If your code fails to run in 3 minutes, you will get no credit for that test.

Submission:

You need to submit the following items.

- `sr_router.c`
- `sr_arpcache.c`
- `readme.pdf`

The gzipped tar file should be named as

"{YourID_without_dash}_{YourNameInEnglish}_assign4". If your student ID is 2024-12345 and your name is "Haechan Kim", then it should be 202412345_HaechanKim_assign4.

Note that you should use your student ID without the dash (-). Assume you have the files in the current directory:

```
$ mkdir 202412345_HaechanKim_assign4
```

```
$ cp sr_router.c sr_arpcache.c readme.pdf 202412345_HaechanKim_assign4
```



```
$ tar zcf 202412345_HaechanKim_assign4.tar.gz 202412345_HaechanKim_assign4
```

Late Policy:

- We will not accept late submission. Any late submission will result in zero credit. We may consider extension if you are in extraordinary circumstances (e.g., hospitalization). Please inform us as soon as you know.

Collaboration Policy:

- You can discuss with other students, but you should write the code on your own – you should **NOT** look at someone else's code nor show/give your code to anyone else. You can view and/or study generic network socket programming code or the code that explains the usage of a system/socket call.
- Please stay away from any kind of plagiarism. If you are not sure about anything, ask TAs or the instructor.

Where can I ask questions?

We encourage students to actively participate in the course. Please share your experiences and questions in the course Classum group. However, if you think your question is not appropriate for sharing for some reason (such as it contains private information), send it to TA (cerotyki@gmail.com). However, TA may share your question on Classum.

Q&A

Q. Can I modify the skeleton code?

A. You can add some variables to three target functions, but **do not modify any skeleton code**. If necessary, ask the TA about such modification!

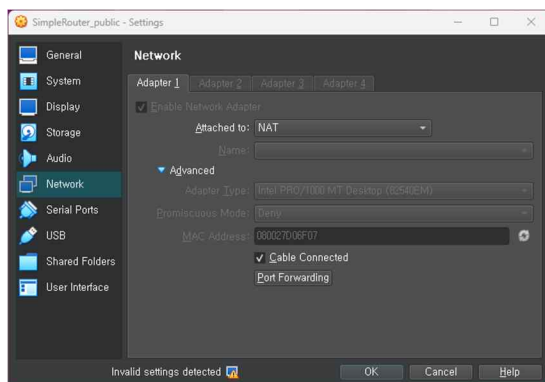
Q. Can I change VirtualBox settings?

A. Yes, you can, but on your responsibility, although changing memory size or CPU core numbers would not affect much.

Q. How can I connect to the VM from the host?

A. In VirtualBox, configure the port-forward setting.

Click “Settings” → “Network” → “Adapter 1”.



The first adapter will use NAT by default. If not, change it to NAT.

Then, click “Advanced” → “Port Forwarding”, and add a port forwarding rule as below.

A screenshot of the 'Port Forwarding Rules' window. It contains a table with one rule, 'Rule 1'.

Rule	Name	Protocol	Host IP	Host Port	Guest IP	Guest Port
Rule 1		TCP	0.0.0.0	2224	10.0.2.15	22

2224 is an example port, and you can use another port number.

10.0.2.15 means your VM's default IP.

Now, you can login to the VM using ssh on windows terminal (use CMD or powershell, no WSL at this point).

For ssh, use the port number you set above.

```
PS C:\Users\junghan> ssh student@127.0.0.1 -p 2224
student@127.0.0.1's password:
Welcome to Ubuntu 22.04.3 LTS (GNU/Linux 5.15.0-94-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

13 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

The list of available updates is more than a week old.
To check for new updates run: sudo apt update
Last login: Wed May 22 20:00:39 2024 from 10.0.2.2
student@SimpleRouter:~$
```

Q. How can I connect to the VM from WSL?

A. Use the IP address of your windows PC, which is accessible by WSL.

First of all, you should know the IP address of your windows PC which is accessible by WSL. On WSL, type below to query the default gateway IP.

```
→ junghan ip route list
default via 172.31.144.1 dev eth0 proto kernel
172.31.144.0/20 dev eth0 proto kernel scope link src 172.31.152.69
```

As shown above, the IP address of my windows OS seen by the WSL is 172.31.144.1.

Then, I can access the VM on WSL by accessing to the IP address.

```
→ junghan ssh -p2224 student@172.31.144.1 -X
Warning: No xauth data; using fake authentication data for X11 forwarding.
Welcome to Ubuntu 22.04.4 LTS (GNU/Linux 5.15.0-141-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

113 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

24 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

New release '24.04.2 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

*** System restart required ***
Last login: Mon Jun 23 18:03:29 2025 from 10.0.2.2
student@SimpleRouter:~$
```

Note that -X option is used for x11 forwarding to WSL. Now, I can open wireshark GUI on WSL.

Q. How can I edit the code (what editors can I use)?

A. You can use any editor that you want. For example, it can be Vim or Emacs in the VM, or you can remotely connect other editors such as Visual Studio Code.

Q. How can I remotely connect using VScode?

A. If you set the network setting, your VScode can connect to the VM using the same ssh port number you set above (e.g., 2224).

Q. How can I download the file from the VM?

A. The easiest way is using drag-and-drop from VirtualBox to your operating system. If you can't, the alternative would be setting a shared folder at VirtualBox to connect to your operating system. If you still have an issue, use "sftp" or "scp" with the ssh port number you set above (e.g., 2224).

Q. There are some IPv6 packets shown in the POX shell. What are they?

A. You do not need to be afraid of them. They are just for connecting and maintaining the connection between Mininet and your Simple Router. Ignore them. Use other methods for debugging rather than POX shell outputs.

Q. There are some unused entries in `struct st_icmp_t3_hdr`. How should I fill this?

A. If you are passing (copying) the packet, you should follow the original packet. However, if you create a new packet, fill them as zero.